

KNN IMPUTATION MODELING NOTEBOOK

Football Match Outcome Prediction

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.impute import KNNImputer
from sklearn.preprocessing import StandardScaler

import warnings
warnings.filterwarnings("ignore")

print("Libraries imported successfully.")
```

```
home_team = pd.read_csv("train_home_team_statistics_df.csv")
home_team.head()
```

ID	LEAGUE	TEAM_NAME
0	Ligue 1	Toulouse
1	Ligue 2	Brest
2	Serie A	Sampdoria
3	League One	Coventry City
4	Premier League	Wolverhampton Wanderers

TEAM_SHOTS_INSIDEBOX_season_sum	TEAM_SHOTS_OFF_TARGET_season_sum
2.0	5.0
8.0	3.0

2	2.0	5.0
3	5.0	5.0
4	3.0	2.0

TEAM_SHOTS_ON_TARGET_season_sum		
TEAM_SHOTS_OUTSIDEBOX_season_sum \		
0	2.0	1.0
1	6.0	5.0
2	2.0	8.0
3	6.0	6.0
4	3.0	4.0

TEAM_PASSES_season_sum		
TEAM_SUCCESSFUL_PASSES_season_sum ... \		
0	2.0	2.0 ...
1	8.0	7.0 ...
2	1.0	1.0 ...
3	9.0	9.0 ...
4	4.0	3.0 ...

TEAM_YELLOWCARDS_5_last_match_std		
TEAM_REDCARDS_5_last_match_std \		
0	3.0	0.0
1	4.0	0.0
2	4.0	5.0
3	4.0	0.0
4	1.0	0.0

TEAM_OFFSIDES_5_last_match_std		
TEAM_ATTACKS_5_last_match_std \		
0	6.0	0.0
1	4.0	3.0
2	6.0	3.0
3	1.0	8.0
4	2.0	5.0

TEAM_PENALTIES_5_last_match_std		
TEAM_SUBSTITUTIONS_5_last_match_std \		
0	10.0	

8.0	
1	10.0
0.0	
2	6.0
7.0	
3	8.0
5.0	
4	8.0
7.0	

TEAM_BALL_SAFE_5_last_match_std	
TEAM_DANGEROUS_ATTACKS_5_last_match_std \	
0	7.0
2.0	
1	1.0
2.0	
2	2.0
3.0	
3	5.0
5.0	
4	2.0
6.0	

TEAM_INJURIES_5_last_match_std		TEAM_GOALS_5_last_match_std	
0	4.0		3.0
1	8.0		4.0
2	2.0		4.0
3	NaN		6.0
4	4.0		4.0

[5 rows x 143 columns]

```
away_team = pd.read_csv("train_away_team_statistics_df.csv")
away_team.head()
```

ID	LEAGUE	TEAM_NAME
TEAM_SHOTS_TOTAL_season_sum \		
0	0	Ligue 1 Olympique Marseille
4.0		
1	1	Ligue 2 Red Star
4.0		
2	2	Serie A Bologna
4.0		
3	3	League One Bolton Wanderers
0.0		
4	4	Premier League Leicester City
5.0		

TEAM_SHOTS_INSIDEBOX_season_sum	
TEAM_SHOTS_OFF_TARGET_season_sum \	

0	1.0	4.0
1	3.0	4.0
2	3.0	4.0
3	0.0	0.0
4	4.0	5.0

TEAM_SHOTS_ON_TARGET_season_sum		
TEAM_SHOTS_OUTSIDEBOX_season_sum \		
0	1.0	5.0
1	3.0	8.0
2	2.0	6.0
3	0.0	1.0
4	6.0	5.0

TEAM_PASSES_season_sum		
TEAM_SUCCESSFUL_PASSES_season_sum ... \		
0	4.0	3.0 ...
1	8.0	7.0 ...
2	4.0	4.0 ...
3	3.0	3.0 ...
4	6.0	6.0 ...

TEAM_YELLOWCARDS_5_last_match_std		
TEAM_REDCARDS_5_last_match_std \		
0	5.0	5.0
1	0.0	0.0
2	6.0	10.0
3	0.0	0.0
4	1.0	0.0

TEAM_OFFSIDES_5_last_match_std		
TEAM_ATTACKS_5_last_match_std \		
0	4.0	0.0
1	3.0	1.0
2	4.0	4.0
3	1.0	2.0
4	4.0	4.0

	TEAM_PENALTIES_5_last_match_std	TEAM_SUBSTITUTIONS_5_last_match_std \
0		6.0
8.0		
1		8.0
4.0		
2		0.0
8.0		
3		0.0
5.0		
4		9.0
4.0		

	TEAM_BALL_SAFE_5_last_match_std	TEAM_DANGEROUS_ATTACKS_5_last_match_std \
0		4.0
3.0		
1		10.0
0.0		
2		3.0
0.0		
3		6.0
3.0		
4		1.0
4.0		

	TEAM_INJURIES_5_last_match_std	TEAM_GOALS_5_last_match_std
0	2.0	3.0
1	5.0	3.0
2	9.0	6.0
3	NaN	2.0
4	6.0	5.0

[5 rows x 143 columns]

```
home_player = pd.read_csv("train_home_player_statistics_df.csv")
home_player.head()
```

	ID	LEAGUE	TEAM_NAME	POSITION	PLAYER_NAME \
0	0	Ligue 1	Toulouse	defender	Agustín Rogel
1	0	Ligue 1	Toulouse	defender	Mathieu Goncalves
2	0	Ligue 1	Toulouse	goalkeeper	Baptiste Reynet
3	0	Ligue 1	Toulouse	midfielder	Jean-Victor Makengo
4	0	Ligue 1	Toulouse	NaN	Efthymios Koulouris

	PLAYER_ACCURATE_CROSSES_season_sum	PLAYER_ACCURATE_PASSES_season_sum \
0		0.0
8.0		
1		0.0

7.0	
2	0.0
33.0	
3	8.0
27.0	
4	2.0
23.0	

	PLAYER_AERIALS_WON_season_sum	PLAYER_ASSISTS_season_sum	\
0	5.0	0.0	
1	4.0	0.0	
2	2.0	0.0	
3	2.0	0.0	
4	27.0	14.0	

	PLAYER_BIG_CHANCES_CREATED_season_sum	...	\
0	0.0	...	
1	0.0	...	
2	0.0	...	
3	8.0	...	
4	8.0	...	

	PLAYER_STARTING_LINEUP_5_last_match_std	\
0	NaN	
1	63.0	
2	0.0	
3	63.0	
4	63.0	

	PLAYER_SUCCESSFUL_DRIBBLES_5_last_match_std	\
0	NaN	
1	14.0	
2	0.0	
3	0.0	
4	17.0	

	PLAYER_TACKLES_5_last_match_std	
	PLAYER_TOTAL_CROSSES_5_last_match_std	\
0	NaN	
NaN		
1	13.0	
0.0		
2	0.0	
0.0		
3	13.0	
6.0		
4	10.0	
13.0		

	PLAYER_TOTAL_DUELS_5_last_match_std
--	-------------------------------------

PLAYER_YELLOWCARDS_5_last_match_std \	
0	NaN
NaN	
1	36.0
77.0	
2	3.0
0.0	
3	17.0
0.0	
4	27.0
0.0	

PLAYER_PUNCHES_5_last_match_std PLAYER_LONG_BALLS_5_last_match_std		
\		
0	NaN	NaN
1	NaN	NaN
2	NaN	NaN
3	NaN	NaN
4	NaN	NaN

PLAYER_LONG_BALLS_WON_5_last_match_std \	
0	NaN
1	NaN
2	NaN
3	NaN
4	NaN

PLAYER_SHOTS_OFF_TARGET_5_last_match_std	
0	NaN
1	NaN
2	NaN
3	NaN
4	NaN

[5 rows x 307 columns]

```
away_player = pd.read_csv("train_away_player_statistics_df.csv")
away_player.head()
```

ID	LEAGUE	TEAM_NAME	POSITION	PLAYER_NAME
\				
0	0	Ligue 1	Olympique Marseille	defender Lucas Perrin
1	0	Ligue 1	Olympique Marseille	midfielder Kevin Strootman
2	0	Ligue 1	Olympique Marseille	goalkeeper Yohann Pelé

3	0	Ligue 1	Olympique Marseille	defender	Abdallah Ali Mohamed
4	0	Ligue 1	Olympique Marseille	NaN	Nemanja Radonjic

PLAYER_ACCURATE_CROSSES_season_sum	
PLAYER_ACCURATE_PASSES_season_sum \	
0	0.0
19.0	
1	2.0
48.0	
2	0.0
0.0	
3	0.0
0.0	
4	8.0
11.0	

PLAYER_AERIALS_WON_season_sum	PLAYER_ASSISTS_season_sum \
0	20.0 0.0
1	19.0 28.0
2	0.0 0.0
3	0.0 0.0
4	4.0 0.0

PLAYER_BIG_CHANCES_CREATED_season_sum ... \	
0	0.0 ...
1	8.0 ...
2	0.0 ...
3	0.0 ...
4	0.0 ...

PLAYER_STARTING_LINEUP_5_last_match_std \	
0	70.0
1	77.0
2	0.0
3	NaN
4	77.0

PLAYER_SUCCESSFUL_DRIBBLES_5_last_match_std \	
0	30.0
1	0.0
2	0.0
3	NaN
4	69.0

PLAYER_TACKLES_5_last_match_std	
PLAYER_TOTAL_CROSSES_5_last_match_std \	
0	46.0

0.0	
1	40.0
6.0	
2	0.0
0.0	
3	NaN
NaN	
4	21.0
13.0	

PLAYER_TOTAL_DUELS_5_last_match_std	
PLAYER_YELLOWCARDS_5_last_match_std \	
0	79.0
99.0	
1	46.0
77.0	
2	0.0
0.0	
3	NaN
NaN	
4	66.0
0.0	

PLAYER_PUNCHES_5_last_match_std		PLAYER_LONG_BALLS_5_last_match_std	
\			
0	NaN		NaN
1	NaN		NaN
2	NaN		NaN
3	NaN		NaN
4	NaN		NaN

PLAYER_LONG_BALLS_WON_5_last_match_std \	
0	NaN
1	NaN
2	NaN
3	NaN
4	NaN

PLAYER_SHOTS_OFF_TARGET_5_last_match_std	
0	NaN
1	NaN
2	NaN
3	NaN
4	NaN

```
[5 rows x 307 columns]
```

```
y_train = pd.read_csv("Y_train_1rknArQ.csv")
y_train.head()
```

	ID	HOME_WINS	DRAW	AWAY_WINS
0	0	0	0	1
1	1	0	1	0
2	2	0	0	1
3	3	1	0	0
4	4	0	1	0

```
y_train_supp = pd.read_csv("Y_train_supp.csv")
y_train_supp.head()
```

	ID	GOAL_DIFF_HOME_AWAY
0	0	-2.0
1	1	0.0
2	2	-1.0
3	3	1.0
4	4	0.0

```
# Understanding the Data
```

```
def dataset_summary(df, name):
    print("=" * 80)
    print(f"{name}")
    print("=" * 80)

    print(f"\nRows      : {df.shape[0]:,}")
    print(f"Columns   : {df.shape[1]:,}")

    print("\nFirst 5 rows")
    display(df.head())

    print("\nData Types")
    display(df.dtypes.value_counts())

    print("\nMemory Usage")

    memory = df.memory_usage(deep=True).sum() / (1024**2)

    print(f"{memory:.2f} MB")

dataset_summary(home_team, "HOME TEAM")
dataset_summary(away_team, "AWAY TEAM")
dataset_summary(home_player, "HOME PLAYER")
dataset_summary(away_player, "AWAY PLAYER")
```

```
dataset_summary(y_train, "Y TRAIN")
```

```
dataset_summary(y_train_supp, "Y TRAIN SUPPLEMENT")
```

```
=====
=====
HOME TEAM
=====
=====
```

```
Rows      : 12,303
```

```
Columns   : 143
```

```
First 5 rows
```

ID	LEAGUE	TEAM_NAME
0	Ligue 1	Toulouse
1	Ligue 2	Brest
2	Serie A	Sampdoria
3	League One	Coventry City
4	Premier League	Wolverhampton Wanderers

TEAM_SHOTS_INSIDEBOX_season_sum	TEAM_SHOTS_OFF_TARGET_season_sum \
0	2.0 5.0
1	8.0 3.0
2	2.0 5.0
3	5.0 5.0
4	3.0 2.0

TEAM_SHOTS_ON_TARGET_season_sum	TEAM_SHOTS_OUTSIDEBOX_season_sum \
0	2.0 1.0
1	6.0 5.0
2	2.0 8.0
3	6.0 6.0

4	3.0	4.0
---	-----	-----

	TEAM_PASSES_season_sum	TEAM_SUCCESSFUL_PASSES_season_sum	...	\
0	2.0	2.0	...	
1	8.0	7.0	...	
2	1.0	1.0	...	
3	9.0	9.0	...	
4	4.0	3.0	...	

	TEAM_YELLOWCARDS_5_last_match_std	
	TEAM_REDCARDS_5_last_match_std	\
0	3.0	0.0
1	4.0	0.0
2	4.0	5.0
3	4.0	0.0
4	1.0	0.0

	TEAM_OFFSIDES_5_last_match_std	TEAM_ATTACKS_5_last_match_std	\
0	6.0	0.0	
1	4.0	3.0	
2	6.0	3.0	
3	1.0	8.0	
4	2.0	5.0	

	TEAM_PENALTIES_5_last_match_std
	TEAM_SUBSTITUTIONS_5_last_match_std
0	10.0
8.0	
1	10.0
0.0	
2	6.0
7.0	
3	8.0
5.0	
4	8.0
7.0	

	TEAM_BALL_SAFE_5_last_match_std
	TEAM_DANGEROUS_ATTACKS_5_last_match_std
0	7.0
2.0	
1	1.0
2.0	

```
2
3.0
3
5.0
4
6.0
```

```
TEAM_INJURIES_5_last_match_std TEAM_GOALS_5_last_match_std
0 4.0 3.0
1 8.0 4.0
2 2.0 4.0
3 NaN 6.0
4 4.0 4.0
```

[5 rows x 143 columns]

Data Types

```
float64    140
object      2
int64       1
Name: count, dtype: int64
```

Memory Usage

14.64 MB

```
=====
=====
AWAY TEAM
=====
=====
```

```
Rows      : 12,303
Columns   : 143
```

First 5 rows

```
   ID  LEAGUE  TEAM_NAME
TEAM_SHOTS_TOTAL_season_sum \
0  0  Ligue 1  Olympique Marseille
4.0
1  1  Ligue 2  Red Star
4.0
2  2  Serie A  Bologna
4.0
3  3  League One  Bolton Wanderers
0.0
4  4  Premier League  Leicester City
5.0
```

TEAM_SHOTS_INSIDEBOX_season_sum TEAM_SHOTS_OFF_TARGET_season_sum \		
0	1.0	4.0
1	3.0	4.0
2	3.0	4.0
3	0.0	0.0
4	4.0	5.0

TEAM_SHOTS_ON_TARGET_season_sum TEAM_SHOTS_OUTSIDEBOX_season_sum \		
0	1.0	5.0
1	3.0	8.0
2	2.0	6.0
3	0.0	1.0
4	6.0	5.0

TEAM_PASSES_season_sum TEAM_SUCCESSFUL_PASSES_season_sum ... \				
0	4.0	3.0	...	
1	8.0	7.0	...	
2	4.0	4.0	...	
3	3.0	3.0	...	
4	6.0	6.0	...	

TEAM_YELLOWCARDS_5_last_match_std TEAM_REDCARDS_5_last_match_std \		
0	5.0	5.0
1	0.0	0.0
2	6.0	10.0
3	0.0	0.0
4	1.0	0.0

TEAM_OFFSIDES_5_last_match_std TEAM_ATTACKS_5_last_match_std \		
0	4.0	0.0
1	3.0	1.0
2	4.0	4.0
3	1.0	2.0

```
4                                4.0                                4.0
```

```
TEAM_PENALTIES_5_last_match_std
TEAM_SUBSTITUTIONS_5_last_match_std \
0                                6.0
8.0
1                                8.0
4.0
2                                0.0
8.0
3                                0.0
5.0
4                                9.0
4.0
```

```
TEAM_BALL_SAFE_5_last_match_std
TEAM_DANGEROUS_ATTACKS_5_last_match_std \
0                                4.0
3.0
1                                10.0
0.0
2                                3.0
0.0
3                                6.0
3.0
4                                1.0
4.0
```

```
TEAM_INJURIES_5_last_match_std TEAM_GOALS_5_last_match_std
0                                2.0                                3.0
1                                5.0                                3.0
2                                9.0                                6.0
3                                NaN                                 2.0
4                                6.0                                5.0
```

```
[5 rows x 143 columns]
```

Data Types

```
float64    140
object      2
int64       1
Name: count, dtype: int64
```

Memory Usage

```
14.64 MB
```

```
=====
```

```
=====
```

```
HOME PLAYER
```

```
=====
=====
Rows      : 237,079
Columns   : 307
```

First 5 rows

	ID	LEAGUE	TEAM_NAME	POSITION	PLAYER_NAME \
0	0	Ligue 1	Toulouse	defender	Agustín Rogel
1	0	Ligue 1	Toulouse	defender	Mathieu Goncalves
2	0	Ligue 1	Toulouse	goalkeeper	Baptiste Reynet
3	0	Ligue 1	Toulouse	midfielder	Jean-Victor Makengo
4	0	Ligue 1	Toulouse	NaN	Efthymios Koulouris

	PLAYER_ACCURATE_CROSSES_season_sum
0	0.0
1	8.0
2	7.0
3	33.0
4	27.0

	PLAYER_ACCURATE_PASSES_season_sum \
0	0.0
1	0.0
2	0.0
3	8.0
4	2.0

	PLAYER_AERIALS_WON_season_sum	PLAYER_ASSISTS_season_sum \
0	5.0	0.0
1	4.0	0.0
2	2.0	0.0
3	2.0	0.0
4	27.0	14.0

	PLAYER_BIG_CHANCES_CREATED_season_sum ... \
0	0.0 ...
1	0.0 ...
2	0.0 ...
3	8.0 ...
4	8.0 ...

	PLAYER_STARTING_LINEUP_5_last_match_std \
0	NaN
1	63.0
2	0.0
3	63.0
4	63.0

PLAYER_SUCCESSFUL_DRIBBLES_5_last_match_std \

0	NaN
1	14.0
2	0.0
3	0.0
4	17.0

PLAYER_TACKLES_5_last_match_std	
PLAYER_TOTAL_CROSSES_5_last_match_std \	
0	NaN
NaN	
1	13.0
0.0	
2	0.0
0.0	
3	13.0
6.0	
4	10.0
13.0	

PLAYER_TOTAL_DUELS_5_last_match_std	
PLAYER_YELLOWCARDS_5_last_match_std \	
0	NaN
NaN	
1	36.0
77.0	
2	3.0
0.0	
3	17.0
0.0	
4	27.0
0.0	

PLAYER_PUNCHES_5_last_match_std		PLAYER_LONG_BALLS_5_last_match_std
\		
0	NaN	NaN
1	NaN	NaN
2	NaN	NaN
3	NaN	NaN
4	NaN	NaN

PLAYER_LONG_BALLS_WON_5_last_match_std \	
0	NaN
1	NaN
2	NaN
3	NaN

4	NaN
PLAYER_SHOTS_OFF_TARGET_5_last_match_std	
0	NaN
1	NaN
2	NaN
3	NaN
4	NaN

[5 rows x 307 columns]

Data Types

```
float64    302
object      4
int64       1
Name: count, dtype: int64
```

Memory Usage

601.93 MB

```
=====
=====
AWAY PLAYER
=====
=====
```

```
Rows      : 236,132
Columns   : 307
```

First 5 rows

	ID	LEAGUE	TEAM_NAME	POSITION	PLAYER_NAME
\	0	0	Ligue 1 Olympique Marseille	defender	Lucas Perrin
	1	0	Ligue 1 Olympique Marseille	midfielder	Kevin Strootman
	2	0	Ligue 1 Olympique Marseille	goalkeeper	Yohann Pelé
	3	0	Ligue 1 Olympique Marseille	defender	Abdallah Ali Mohamed
	4	0	Ligue 1 Olympique Marseille	NaN	Nemanja Radonjic

PLAYER_ACCURATE_CROSSES_season_sum	
PLAYER_ACCURATE_PASSES_season_sum	\
0	0.0
19.0	
1	2.0

48.0

2 0.0

0.0

3 0.0

0.0

4 8.0

11.0

PLAYER_AERIALS_WON_season_sum PLAYER_ASSISTS_season_sum \

0 20.0 0.0

1 19.0 28.0

2 0.0 0.0

3 0.0 0.0

4 4.0 0.0

PLAYER_BIG_CHANCES_CREATED_season_sum ... \

0 0.0 ...

1 8.0 ...

2 0.0 ...

3 0.0 ...

4 0.0 ...

PLAYER_STARTING_LINEUP_5_last_match_std \

0 70.0

1 77.0

2 0.0

3 NaN

4 77.0

PLAYER_SUCCESSFUL_DRIBBLES_5_last_match_std \

0 30.0

1 0.0

2 0.0

3 NaN

4 69.0

PLAYER_TACKLES_5_last_match_std

PLAYER_TOTAL_CROSSES_5_last_match_std \

0 46.0

0.0

1 40.0

6.0

2 0.0

0.0

3 NaN

NaN

4 21.0

13.0

PLAYER_TOTAL_DUELS_5_last_match_std

	PLAYER_YELLOWCARDS_5_last_match_std \
0	79.0
1	46.0
2	0.0
3	NaN
4	66.0

	PLAYER_PUNCHES_5_last_match_std	PLAYER_LONG_BALLS_5_last_match_std
0	NaN	NaN
1	NaN	NaN
2	NaN	NaN
3	NaN	NaN
4	NaN	NaN

	PLAYER_LONG_BALLS_WON_5_last_match_std \
0	NaN
1	NaN
2	NaN
3	NaN
4	NaN

	PLAYER_SHOTS_OFF_TARGET_5_last_match_std
0	NaN
1	NaN
2	NaN
3	NaN
4	NaN

[5 rows x 307 columns]

Data Types

```
float64    302
object      4
int64       1
Name: count, dtype: int64
```

Memory Usage

599.54 MB

=====

Y TRAIN

=====

=====

Rows : 12,303
Columns : 4

First 5 rows

	ID	HOME_WINS	DRAW	AWAY_WINS
0	0	0	0	1
1	1	0	1	0
2	2	0	0	1
3	3	1	0	0
4	4	0	1	0

Data Types

int64 4
Name: count, dtype: int64

Memory Usage
0.38 MB

=====

Y TRAIN SUPPLEMENT

=====

=====

Rows : 12,303
Columns : 2

First 5 rows

	ID	GOAL_DIFF_HOME_AWAY
0	0	-2.0
1	1	0.0
2	2	-1.0
3	3	1.0
4	4	0.0

Data Types

```
int64      1
float64     1
Name: count, dtype: int64
```

```
Memory Usage
0.19 MB
```

```
# Check dataset shapes
```

```
datasets = {
    "home_team": home_team,
    "away_team": away_team,
    "home_player": home_player,
    "away_player": away_player,
    "y_train": y_train,
    "y_train_supp": y_train_supp
}

for name, df in datasets.items():
    print(f"{name:15s}: {df.shape}")
```

```
home_team      : (12303, 143)
away_team      : (12303, 143)
home_player    : (237079, 307)
away_player    : (236132, 307)
y_train        : (12303, 4)
y_train_supp   : (12303, 2)
```

```
# Check missing values in each dataset
```

```
for name, df in datasets.items():
    print(f"\n{'=' * 60}")
    print(name)
    print(f"{'=' * 60}")

    missing = df.isnull().sum()
    missing = missing[missing > 0].sort_values(ascending=False)

    if len(missing) == 0:
        print("No missing values.")
    else:
        display(missing.to_frame("Missing Values"))
```

```
=====
home_team
=====
```

	Missing Values
TEAM_INJURIES_5_last_match_std	3318

TEAM_INJURIES_5_last_match_sum	2932
TEAM_INJURIES_5_last_match_average	2931
TEAM_INJURIES_season_std	2924
TEAM_INJURIES_season_sum	1960
...	...
TEAM_YELLOWCARDS_5_last_match_std	1
TEAM_CORNERS_5_last_match_average	1
TEAM_YELLOWCARDS_season_std	1
TEAM_CORNERS_season_sum	1
TEAM_CORNERS_5_last_match_sum	1

[118 rows x 1 columns]

=====
away_team
=====

	Missing Values
TEAM_INJURIES_5_last_match_std	3313
TEAM_INJURIES_5_last_match_sum	2933
TEAM_INJURIES_5_last_match_average	2932
TEAM_INJURIES_season_std	2926
TEAM_INJURIES_season_sum	1961
...	...
TEAM_YELLOWCARDS_5_last_match_std	1
TEAM_CORNERS_5_last_match_average	1
TEAM_YELLOWCARDS_season_std	1
TEAM_CORNERS_season_sum	1
TEAM_CORNERS_5_last_match_sum	1

[118 rows x 1 columns]

=====
home_player
=====

	Missing Values
PLAYER_SHOTS_OFF_TARGET_season_std	237079
PLAYER_LONG_BALLS_5_last_match_sum	237079
PLAYER_SHOTS_OFF_TARGET_season_sum	237079
PLAYER_CAPTAIN_season_average	237079
PLAYER_LONG_BALLS_season_average	237079
...	...
PLAYER_ASSISTS_season_sum	3597
PLAYER_ASSISTS_season_average	3597
PLAYER_MINUTES_PLAYED_5_last_match_sum	3597
PLAYER_GOALS_season_sum	3597
PLAYER_GOALS_season_average	3597

```
[303 rows x 1 columns]
```

```
=====
away_player
=====
```

	Missing Values
PLAYER_SHOTS_OFF_TARGET_season_std	236132
PLAYER_LONG_BALLS_5_last_match_sum	236132
PLAYER_SHOTS_OFF_TARGET_season_sum	236132
PLAYER_CAPTAIN_season_average	236132
PLAYER_LONG_BALLS_season_average	236132
...	...
PLAYER_ASSISTS_season_sum	3829
PLAYER_ASSISTS_season_average	3829
PLAYER_MINUTES_PLAYED_5_last_match_sum	3829
PLAYER_GOALS_season_sum	3829
PLAYER_GOALS_season_average	3829

```
[303 rows x 1 columns]
```

```
=====
y_train
=====
No missing values.

=====
y_train_supp
=====
No missing values.
```

```
# Remove columns that contain 100% missing values
```

```
def remove_all_missing_columns(df, name):
    before = df.shape[1]

    df = df.dropna(axis=1, how="all")

    after = df.shape[1]

    print(f"{name}:")
    print(f"Columns before : {before}")
    print(f"Columns after  : {after}")
    print(f"Columns removed: {before - after}")
    print()

    return df
```

```
home_team = remove_all_missing_columns(home_team, "HOME TEAM")
away_team = remove_all_missing_columns(away_team, "AWAY TEAM")
home_player = remove_all_missing_columns(home_player, "HOME PLAYER")
away_player = remove_all_missing_columns(away_player, "AWAY PLAYER")
```

HOME TEAM:

```
Columns before : 143
Columns after  : 143
Columns removed: 0
```

AWAY TEAM:

```
Columns before : 143
Columns after  : 143
Columns removed: 0
```

HOME PLAYER:

```
Columns before : 307
Columns after  : 285
Columns removed: 22
```

AWAY PLAYER:

```
Columns before : 307
Columns after  : 285
Columns removed: 22
```

```
print("Shapes after removing completely missing columns:")
```

```
print(f"home_team    : {home_team.shape}")
print(f"away_team    : {away_team.shape}")
print(f"home_player   : {home_player.shape}")
print(f"away_player   : {away_player.shape}")
```

Shapes after removing completely missing columns:

```
home_team    : (12303, 143)
away_team    : (12303, 143)
home_player   : (237079, 285)
away_player   : (236132, 285)
```

```
# Identify categorical and numerical columns
```

```
for name, df in {
    "home_team": home_team,
    "away_team": away_team,
    "home_player": home_player,
    "away_player": away_player
}.items():
```

```
    categorical = df.select_dtypes(include=["object",
"category"]).columns.tolist()
    numerical = df.select_dtypes(include=["number"]).columns.tolist()
```

```

print(f"\n{name}")
print("-" * 60)
print(f"Categorical columns : {len(categorical)}")
print(f"Numerical columns   : {len(numerical)}")
print(f"Categorical         : {categorical}")

```

home_team

```

-----
Categorical columns : 2
Numerical columns   : 141
Categorical         : ['LEAGUE', 'TEAM_NAME']

```

away_team

```

-----
Categorical columns : 2
Numerical columns   : 141
Categorical         : ['LEAGUE', 'TEAM_NAME']

```

home_player

```

-----
Categorical columns : 4
Numerical columns   : 281
Categorical         : ['LEAGUE', 'TEAM_NAME', 'POSITION',
'PLAYER_NAME']

```

away_player

```

-----
Categorical columns : 4
Numerical columns   : 281
Categorical         : ['LEAGUE', 'TEAM_NAME', 'POSITION',
'PLAYER_NAME']

```

Separate numerical columns for KNN imputation

```

home_team_num = home_team.select_dtypes(include=["number"]).copy()
away_team_num = away_team.select_dtypes(include=["number"]).copy()

```

```

home_player_num = home_player.select_dtypes(include=["number"]).copy()
away_player_num = away_player.select_dtypes(include=["number"]).copy()

```

```

print("Numerical dataset shapes:")
print(f"home_team_num   : {home_team_num.shape}")
print(f"away_team_num   : {away_team_num.shape}")
print(f"home_player_num  : {home_player_num.shape}")
print(f"away_player_num  : {away_player_num.shape}")

```

```

Numerical dataset shapes:
home_team_num   : (12303, 141)
away_team_num   : (12303, 141)

```

```

home_player_num : (237079, 281)
away_player_num : (236132, 281)

# Count missing numerical values before KNN imputation

numerical_datasets = {
    "home_team": home_team_num,
    "away_team": away_team_num,
    "home_player": home_player_num,
    "away_player": away_player_num
}

for name, df in numerical_datasets.items():
    missing_count = df.isnull().sum().sum()

    print(f"{name}:")
    print(f"    Missing numerical values: {missing_count:,}")
    print(f"    Percentage missing: {missing_count / df.size * 100:.2f}%")
    print()

home_team:
    Missing numerical values: 82,407
    Percentage missing: 4.75%

away_team:
    Missing numerical values: 82,495
    Percentage missing: 4.76%

home_player:
    Missing numerical values: 8,835,889
    Percentage missing: 13.26%

away_player:
    Missing numerical values: 8,883,326
    Percentage missing: 13.39%

# Check whether ID is included in the numerical datasets

for name, df in numerical_datasets.items():
    print(f"{name}:")
    print(f"ID included: {'ID' in df.columns}")
    print()

home_team:
ID included: True

away_team:
ID included: True

```

```
home_player:
ID included: True
```

```
away_player:
ID included: True
```

```
# Keep ID separately for merging later
```

```
home_team_id = home_team["ID"].copy()
away_team_id = away_team["ID"].copy()
```

```
home_player_id = home_player["ID"].copy()
away_player_id = away_player["ID"].copy()
```

```
# Remove ID only from the KNN feature datasets
```

```
home_team_num = home_team_num.drop(columns=["ID"])
away_team_num = away_team_num.drop(columns=["ID"])
```

```
home_player_num = home_player_num.drop(columns=["ID"])
away_player_num = away_player_num.drop(columns=["ID"])
```

```
print("ID retained separately for merging.")
print()
print(f"Home team ID    : {home_team_id.shape}")
print(f"Away team ID     : {away_team_id.shape}")
print(f"Home player ID    : {home_player_id.shape}")
print(f"Away player ID    : {away_player_id.shape}")
```

```
ID retained separately for merging.
```

```
Home team ID    : (12303,)
Away team ID    : (12303,)
Home player ID  : (237079,)
Away player ID  : (236132,)
```

```
from sklearn.impute import KNNImputer
from sklearn.preprocessing import StandardScaler
import pandas as pd
import numpy as np
```

```
# KNN imputation function
```

```
from sklearn.impute import KNNImputer
from sklearn.preprocessing import StandardScaler
```

```
def knn_impute_dataset(df, n_neighbors=5):
    scaler = StandardScaler()
```

```

# Standardise the numerical variables
scaled_data = scaler.fit_transform(df)

# Apply KNN imputation
imputer = KNNImputer(n_neighbors=n_neighbors)
imputed_data = imputer.fit_transform(scaled_data)

# Convert back to original scale
imputed_data = scaler.inverse_transform(imputed_data)

return pd.DataFrame(
    imputed_data,
    columns=df.columns,
    index=df.index
)

# Apply KNN imputation to HOME TEAM

print("Starting KNN imputation for HOME TEAM...")

home_team_num_knn = knn_impute_dataset(
    home_team_num,
    n_neighbors=5
)

print("HOME TEAM KNN imputation completed.")
print("Shape:", home_team_num_knn.shape)
print("Missing values:", home_team_num_knn.isna().sum().sum())

Starting KNN imputation for HOME TEAM...
HOME TEAM KNN imputation completed.
Shape: (12303, 140)
Missing values: 0

# Apply KNN imputation to AWAY TEAM

print("Starting KNN imputation for AWAY TEAM...")

away_team_num_knn = knn_impute_dataset(
    away_team_num,
    n_neighbors=5
)

print("AWAY TEAM KNN imputation completed.")
print("Shape:", away_team_num_knn.shape)
print("Missing values:", away_team_num_knn.isna().sum().sum())

Starting KNN imputation for AWAY TEAM...
AWAY TEAM KNN imputation completed.
Shape: (12303, 140)
Missing values: 0

```

```

# Check the Home Player numerical dataset before KNN

print("HOME PLAYER numerical dataset:")
print("Shape:", home_player_num.shape)
print("Missing values:", home_player_num.isna().sum().sum())

HOME PLAYER numerical dataset:
Shape: (237079, 280)
Missing values: 8835889

# Check how many Home Player rows are complete

complete_rows = home_player_num.notna().all(axis=1)

print("Total rows:", len(home_player_num))
print("Complete rows:", complete_rows.sum())
print("Rows with missing values:", (~complete_rows).sum())

Total rows: 237079
Complete rows: 4656
Rows with missing values: 232423

# =====
# HOME PLAYER - KNN IMPUTATION
# SINGLE CELL - CLEAN FINAL OUTPUT
# =====

import numpy as np
import pandas as pd

from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors

# -----
# 1. Store original information
# -----

original_shape = home_player_num.shape
missing_before = home_player_num.isna().sum().sum()

# -----
# 2. Identify complete and incomplete rows
# -----

complete_mask = home_player_num.notna().all(axis=1)

complete_data = home_player_num.loc[complete_mask].copy()
incomplete_data = home_player_num.loc[~complete_mask].copy()

```

```

# -----
# 3. Convert complete data to NumPy
# -----

complete_array = complete_data.to_numpy()

# Column means used temporarily to calculate distances
column_means = complete_data.mean()

# -----
# 4. Standardise complete observations
# -----

scaler_player = StandardScaler()

complete_scaled = scaler_player.fit_transform(
    complete_array
)

# -----
# 5. Fit KNN model
# -----

knn_player = NearestNeighbors(
    n_neighbors=5,
    metric="euclidean",
    n_jobs=-1
)

knn_player.fit(complete_scaled)

# -----
# 6. Create output dataset
# -----

home_player_knn = home_player_num.copy()

incomplete_indices = incomplete_data.index.to_numpy()

batch_size = 500

# -----
# 7. Perform KNN imputation in batches
# -----

for start in range(0, len(incomplete_indices), batch_size):

```

```

end = min(
    start + batch_size,
    len(incomplete_indices)
)

batch_indices = incomplete_indices[start:end]

batch = home_player_num.loc[
    batch_indices
].copy()

# Temporarily fill missing values with column means
batch_filled = batch.fillna(column_means)

# Standardise
batch_scaled = scaler_player.transform(
    batch_filled
)

# Find 5 nearest complete observations
distances, neighbour_indices = knn_player.kneighbors(
    batch_scaled
)

# Retrieve neighbour values
neighbour_values = complete_array[
    neighbour_indices
]

# Average the 5 nearest neighbours
imputed_values = neighbour_values.mean(
    axis=1
)

# Convert batch to NumPy
batch_array = batch.to_numpy()

# Identify originally missing cells
missing_mask = np.isnan(batch_array)

# Replace ONLY missing cells
batch_array[missing_mask] = (

```

```

        imputed_values[missing_mask]
    )

    # Put imputed batch back
    home_player_knn.loc[
        batch_indices,
        home_player_num.columns
    ] = batch_array

# -----
# 8. Final results
# -----

missing_after = home_player_knn.isna().sum().sum()

# -----
# 9. ONE CLEAN OUTPUT
# -----

print("=" * 70)
print("HOME PLAYER KNN IMPUTATION COMPLETED")
print("=" * 70)

print(f"Original shape           : {original_shape}")
print(f"Complete rows              : {len(complete_data)}")
print(f"Incomplete rows            : {len(incomplete_data)}")
print(f"Missing values before      : {missing_before:,}")
print(f"Final shape                 : {home_player_knn.shape}")
print(f"Missing values after       : {missing_after:,}")
print(f"KNN neighbours              : 5")

print("=" * 70)

=====
HOME PLAYER KNN IMPUTATION COMPLETED
=====
Original shape           : (237079, 280)
Complete rows            : 4656
Incomplete rows          : 232423
Missing values before    : 8,835,889
Final shape              : (237079, 280)
Missing values after     : 0
KNN neighbours           : 5
=====

# =====
# AWAY PLAYER - KNN IMPUTATION
# SINGLE CELL - CLEAN FINAL OUTPUT

```

```

# =====

import numpy as np
import pandas as pd

from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors

# -----
# 1. Store original information
# -----

original_shape = away_player_num.shape
missing_before = away_player_num.isna().sum().sum()

# -----
# 2. Identify complete and incomplete rows
# -----

complete_mask = away_player_num.notna().all(axis=1)

complete_data = away_player_num.loc[complete_mask].copy()
incomplete_data = away_player_num.loc[~complete_mask].copy()

# -----
# 3. Convert complete data to NumPy
# -----

complete_array = complete_data.to_numpy()

# Column means used temporarily to calculate distances
column_means = complete_data.mean()

# -----
# 4. Standardise complete observations
# -----

scaler_player = StandardScaler()

complete_scaled = scaler_player.fit_transform(
    complete_array
)

# -----
# 5. Fit KNN model
# -----

```

```

knn_player = NearestNeighbors(
    n_neighbors=5,
    metric="euclidean",
    n_jobs=-1
)

knn_player.fit(complete_scaled)

# -----
# 6. Create output dataset
# -----

away_player_knn = away_player_num.copy()

incomplete_indices = incomplete_data.index.to_numpy()

batch_size = 500

# -----
# 7. Perform KNN imputation in batches
# -----

for start in range(0, len(incomplete_indices), batch_size):
    end = min(
        start + batch_size,
        len(incomplete_indices)
    )

    batch_indices = incomplete_indices[start:end]

    batch = away_player_num.loc[
        batch_indices
    ].copy()

    # Temporarily fill missing values with column means
    batch_filled = batch.fillna(column_means)

    # Standardise
    batch_scaled = scaler_player.transform(
        batch_filled
    )

    # Find 5 nearest complete observations
    distances, neighbour_indices = knn_player.kneighbors(

```

```

        batch_scaled
    )

    # Retrieve neighbour values
    neighbour_values = complete_array[
        neighbour_indices
    ]

    # Average the 5 nearest neighbours
    imputed_values = neighbour_values.mean(
        axis=1
    )

    # Convert batch to NumPy
    batch_array = batch.to_numpy()

    # Identify originally missing cells
    missing_mask = np.isnan(batch_array)

    # Replace ONLY missing cells
    batch_array[missing_mask] = (
        imputed_values[missing_mask]
    )

    # Put imputed batch back
    away_player_knn.loc[
        batch_indices,
        away_player_num.columns
    ] = batch_array

# -----
# 8. Final results
# -----

missing_after = away_player_knn.isna().sum().sum()

# -----
# 9. ONE CLEAN OUTPUT
# -----

print("=" * 70)
print("AWAY PLAYER KNN IMPUTATION COMPLETED")
print("=" * 70)

```

```

print(f"Original shape      : {original_shape}")
print(f"Complete rows      : {len(complete_data)}")
print(f"Incomplete rows    : {len(incomplete_data)}")
print(f"Missing values before : {missing_before:,}")
print(f"Final shape           : {away_player_knn.shape}")
print(f"Missing values after  : {missing_after:,}")
print(f"KNN neighbours        : 5")

```

```

print("=" * 70)

```

```

=====
AWAY PLAYER KNN IMPUTATION COMPLETED
=====

```

```

Original shape      : (236132, 280)
Complete rows      : 4679
Incomplete rows    : 231453
Missing values before : 8,883,326
Final shape        : (236132, 280)
Missing values after  : 0
KNN neighbours      : 5
=====

```

```

# =====
# RESTORE ID + AGGREGATE HOME PLAYER DATA + MERGE
# =====

```

```

import pandas as pd

```

```

print("=" * 70)
print("PREPARING HOME PLAYER DATA FOR MERGING")
print("=" * 70)

```

```

# -----
# 1. Restore ID to the KNN-imputed Home Player data
# -----

```

```

home_player_knn = home_player_knn.copy()

```

```

home_player_knn["ID"] = home_player.loc[
    home_player_knn.index, "ID"
].values

```

```

print("ID restored to Home Player KNN data")
print("Home Player KNN shape:", home_player_knn.shape)
print("Unique match IDs:", home_player_knn["ID"].nunique())

```

```

# -----
# 2. Identify numerical columns
# -----

```

```

player_numeric_cols = home_player_knn.select_dtypes(
    include="number"
).columns.tolist()

# Remove ID from variables to aggregate
player_numeric_cols.remove("ID")

# -----
# 3. Aggregate player statistics by match ID
# -----
# Each match has multiple players.
# We calculate the mean player statistic for each match.

home_player_agg = (
    home_player_knn
    .groupby("ID")[player_numeric_cols]
    .mean()
    .reset_index()
)

print("Home Player data aggregated")
print("Aggregated shape:", home_player_agg.shape)

# -----
# 4. Prepare Team datasets
# -----

home_team_merge = home_team.copy()
away_team_merge = away_team.copy()

# -----
# 5. Merge Home Team + Away Team
# -----

merged_knn = pd.merge(
    home_team_merge,
    away_team_merge,
    on="ID",
    suffixes=("_HOME", "_AWAY"),
    how="inner"
)

# -----
# 6. Merge aggregated Home Player data
# -----

```

```

merged_knn = pd.merge(
    merged_knn,
    home_player_agg,
    on="ID",
    how="left"
)

# -----
# 7. Final checks
# -----

print("=" * 70)
print("HOME PLAYER DATA SUCCESSFULLY MERGED")
print("=" * 70)

print("Final shape      :", merged_knn.shape)
print("Unique match IDs  :", merged_knn["ID"].nunique())
print("Duplicate IDs     :", merged_knn["ID"].duplicated().sum())
print("Missing values    :", merged_knn.isna().sum().sum())

print("=" * 70)

=====
PREPARING HOME PLAYER DATA FOR MERGING
=====
ID restored to Home Player KNN data
Home Player KNN shape: (237079, 281)
Unique match IDs: 12303
Home Player data aggregated
Aggregated shape: (12303, 281)
=====
HOME PLAYER DATA SUCCESSFULLY MERGED
=====
Final shape      : (12303, 565)
Unique match IDs : 12303
Duplicate IDs    : 0
Missing values   : 164902
=====

# Check missing values by column in the merged dataset

missing_by_column = merged_knn.isna().sum()

missing_by_column = missing_by_column[missing_by_column >
0].sort_values(ascending=False)

print("=" * 70)
print("MISSING VALUES IN CURRENT MERGED DATASET")
print("=" * 70)

```

```

print(missing_by_column)

print("\nTotal missing values:", missing_by_column.sum())
print("Number of columns with missing values:",
len(missing_by_column))

=====
MISSING VALUES IN CURRENT MERGED DATASET
=====
TEAM_INJURIES_5_last_match_std_HOME          3318
TEAM_INJURIES_5_last_match_std_AWAY          3313
TEAM_INJURIES_5_last_match_sum_AWAY          2933
TEAM_INJURIES_5_last_match_sum_HOME          2932
TEAM_INJURIES_5_last_match_average_AWAY      2932
...
TEAM_CORNERS_5_last_match_average_HOME        1
TEAM_CORNERS_5_last_match_average_AWAY        1
TEAM_CORNERS_5_last_match_sum_AWAY            1
TEAM_YELLOWCARDS_season_std_AWAY              1
TEAM_BALL_POSSESSION_season_average_HOME      1
Length: 236, dtype: int64

Total missing values: 164902
Number of columns with missing values: 236

# =====
# REBUILD MERGED DATASET USING KNN-IMPUTED TEAM + HOME PLAYER DATA
# =====

print("=" * 70)
print("REBUILDING MERGED DATASET")
print("=" * 70)

# -----
# 1. Restore ID to KNN-imputed Home Team data
# -----

home_team_knn = home_team_num_knn.copy()

home_team_knn["ID"] = home_team.loc[
    home_team_knn.index, "ID"
].values

# -----
# 2. Restore ID to KNN-imputed Away Team data
# -----

away_team_knn = away_team_num_knn.copy()

away_team_knn["ID"] = away_team.loc[

```

```

        away_team_knn.index, "ID"
    ].values

# -----
# 3. Merge KNN-imputed Home Team + Away Team
# -----

merged_knn = pd.merge(
    home_team_knn,
    away_team_knn,
    on="ID",
    how="inner",
    suffixes=("_HOME", "_AWAY")
)

# -----
# 4. Merge aggregated Home Player data
# -----

merged_knn = pd.merge(
    merged_knn,
    home_player_agg,
    on="ID",
    how="left"
)

# -----
# 5. Final checks
# -----

missing_total = merged_knn.isna().sum().sum()

print("=" * 70)
print("MERGED DATASET UPDATED SUCCESSFULLY")
print("=" * 70)

print("Final shape      :", merged_knn.shape)
print("Unique match IDs :", merged_knn["ID"].nunique())
print("Duplicate IDs     :", merged_knn["ID"].duplicated().sum())
print("Missing values    :", missing_total)

print("=" * 70)

=====
REBUILDING MERGED DATASET
=====
=====
MERGED DATASET UPDATED SUCCESSFULLY

```

```
=====
Final shape      : (12303, 561)
Unique match IDs : 12303
Duplicate IDs    : 0
Missing values   : 0
=====
```

```
# =====
# ADD AWAY PLAYER KNN DATA TO MERGED DATASET
# =====
```

```
print("=" * 70)
print("ADDING AWAY PLAYER DATA")
print("=" * 70)
```

```
# -----
# 1. Restore ID to Away Player KNN data
# -----
```

```
away_player_knn = away_player_knn.copy()

away_player_knn["ID"] = away_player.loc[
    away_player_knn.index, "ID"
].values
```

```
print("Away Player ID restored")
print("Away Player KNN shape:", away_player_knn.shape)
print("Unique match IDs:", away_player_knn["ID"].nunique())
```

```
# -----
# 2. Identify numerical player variables
# -----
```

```
away_player_numeric_cols = away_player_knn.select_dtypes(
    include="number"
).columns.tolist()
```

```
away_player_numeric_cols.remove("ID")
```

```
# -----
# 3. Aggregate Away Player statistics by match ID
# -----
```

```
away_player_agg = (
    away_player_knn
    .groupby("ID")[away_player_numeric_cols]
    .mean()
    .reset_index()
)
```

```

print("Away Player data aggregated")
print("Aggregated shape:", away_player_agg.shape)

# -----
# 4. Merge Away Player data
# -----

merged_knn = pd.merge(
    merged_knn,
    away_player_agg,
    on="ID",
    how="left"
)

# -----
# 5. Final checks
# -----

missing_total = merged_knn.isna().sum().sum()

print("=" * 70)
print("AWAY PLAYER DATA SUCCESSFULLY ADDED")
print("=" * 70)

print("Final shape      :", merged_knn.shape)
print("Unique match IDs :", merged_knn["ID"].nunique())
print("Duplicate IDs     :", merged_knn["ID"].duplicated().sum())
print("Missing values    :", missing_total)

print("=" * 70)

```

```

=====
ADDING AWAY PLAYER DATA
=====

```

```

Away Player ID restored
Away Player KNN shape: (236132, 281)
Unique match IDs: 12303
Away Player data aggregated
Aggregated shape: (12303, 281)
=====

```

```

AWAY PLAYER DATA SUCCESSFULLY ADDED
=====

```

```

Final shape      : (12303, 841)
Unique match IDs : 12303
Duplicate IDs     : 0
Missing values    : 0
=====

```

```

# =====
# CREATE FINAL FEATURES (X) AND TARGET (y)
# =====

print("=" * 70)
print("CREATING FEATURES AND TARGET")
print("=" * 70)

# -----
# 1. Create a single target variable
# -----
# 0 = Home Win
# 1 = Draw
# 2 = Away Win

y_train = y_train.copy()

y_train["TARGET"] = y_train[
    ["HOME_WINS", "DRAW", "AWAY_WINS"]
].idxmax(axis=1)

y_train["TARGET"] = y_train["TARGET"].map({
    "HOME_WINS": 0,
    "DRAW": 1,
    "AWAY_WINS": 2
})

# -----
# 2. Check target distribution
# -----

print("Target distribution:")
print(y_train["TARGET"].value_counts().sort_index())

# -----
# 3. Check that IDs match
# -----

feature_ids = set(merged_knn["ID"])
target_ids = set(y_train["ID"])

print("\nFeature IDs :", len(feature_ids))
print("Target IDs   :", len(target_ids))
print("IDs match    :", feature_ids == target_ids)

# -----
# 4. Create X
# -----

```

```
# Remove ID because it is only an identifier  
# and should not be used as a predictive feature.
```

```
X = merged_knn.drop(columns=["ID"]).copy()
```

```
# -----  
# 5. Create y  
# -----  
# Match target rows to the same ID order as X.
```

```
y = (  
    y_train  
    .set_index("ID")  
    .loc[merged_knn["ID"], "TARGET"]  
    .reset_index(drop=True)  
)
```

```
# -----  
# 6. Final checks  
# -----
```

```
print("\n" + "=" * 70)  
print("FEATURES AND TARGET CREATED")  
print("=" * 70)
```

```
print("X shape :", X.shape)  
print("y shape :", y.shape)
```

```
print("\nTarget counts:")  
print(y.value_counts().sort_index())
```

```
print("\nMissing values in X :", X.isna().sum().sum())  
print("Missing values in y :", y.isna().sum())
```

```
print("=" * 70)
```

```
=====
```

```
CREATING FEATURES AND TARGET
```

```
=====
```

```
Target distribution:
```

```
TARGET
```

```
0    5356  
1    3186  
2    3761
```

```
Name: count, dtype: int64
```

```
Feature IDs : 12303  
Target IDs  : 12303  
IDs match   : True
```

```
=====
FEATURES AND TARGET CREATED
=====
```

```
X shape : (12303, 840)
y shape : (12303,)
```

```
Target counts:
```

```
TARGET
```

```
0      5356
```

```
1      3186
```

```
2      3761
```

```
Name: count, dtype: int64
```

```
Missing values in X : 0
```

```
Missing values in y : 0
=====
```

```
# =====
# TRAIN-TEST SPLIT
# =====
```

```
from sklearn.model_selection import train_test_split
```

```
print("=" * 70)
```

```
print("CREATING TRAIN AND TEST SETS")
```

```
print("=" * 70)
```

```
# 80% training, 20% testing
```

```
# Stratify keeps the Home Win / Draw / Away Win proportions similar
```

```
X_train, X_test, y_train_final, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.20,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
print("X_train shape :", X_train.shape)
```

```
print("X_test shape  :", X_test.shape)
```

```
print("y_train shape :", y_train_final.shape)
```

```
print("y_test shape  :", y_test.shape)
```

```
print("\nTraining target distribution:")
```

```
print(y_train_final.value_counts().sort_index())
```

```
print("\nTesting target distribution:")
```

```
print(y_test.value_counts().sort_index())
```

```
print("=" * 70)
```

```
=====
CREATING TRAIN AND TEST SETS
=====
```

```
X_train shape : (9842, 840)
X_test shape  : (2461, 840)
y_train shape : (9842,)
y_test shape  : (2461,)
```

```
Training target distribution:
```

```
TARGET
```

```
0    4284
1    2549
2    3009
```

```
Name: count, dtype: int64
```

```
Testing target distribution:
```

```
TARGET
```

```
0    1072
1     637
2     752
```

```
Name: count, dtype: int64
=====
```

```
# =====
# FEATURE SCALING
# =====
```

```
from sklearn.preprocessing import StandardScaler
```

```
print("=" * 70)
print("STANDARDISING FEATURES")
print("=" * 70)
```

```
# Create scaler
```

```
scaler = StandardScaler()
```

```
# Fit ONLY on training data
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
# Apply the same transformation to test data
```

```
X_test_scaled = scaler.transform(X_test)
```

```
print("X_train_scaled shape :", X_train_scaled.shape)
```

```
print("X_test_scaled shape  :", X_test_scaled.shape)
```

```
print("\nScaling completed successfully.")
```

```
print("=" * 70)
```

```
=====
STANDARDISING FEATURES
=====
```

```
X_train_scaled shape : (9842, 840)
X_test_scaled shape  : (2461, 840)
```

```
Scaling completed successfully.
```

```
=====
```

```
# =====
# COMMON MODEL EVALUATION FUNCTION
# =====
```

```
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    log_loss,
    classification_report,
    confusion_matrix
)

def evaluate_model(model_name, model, X_test, y_test):

    # Predictions
    y_pred = model.predict(X_test)

    # Probabilities
    y_prob = model.predict_proba(X_test)

    # Performance metrics
    accuracy = accuracy_score(y_test, y_pred)

    balanced_acc = balanced_accuracy_score(y_test, y_pred)

    macro_precision = precision_score(
        y_test,
        y_pred,
        average="macro",
        zero_division=0
    )

    macro_recall = recall_score(
        y_test,
        y_pred,
        average="macro",
        zero_division=0
    )

    macro_f1 = f1_score(
```

```

        y_test,
        y_pred,
        average="macro",
        zero_division=0
    )

    roc_auc = roc_auc_score(
        y_test,
        y_prob,
        multi_class="ovr",
        average="macro"
    )

    logloss = log_loss(y_test, y_prob)

    # Print results
    print("=" * 70)
    print(f"{model_name.upper()} - TEST SET RESULTS")
    print("=" * 70)

    print(f"Accuracy:                {accuracy:.4f}")
    print(f"Balanced Accuracy:          {balanced_acc:.4f}")
    print(f"Macro Precision:              {macro_precision:.4f}")
    print(f"Macro Recall:                 {macro_recall:.4f}")
    print(f"Macro F1-score:               {macro_f1:.4f}")
    print(f"ROC-AUC (Macro):              {roc_auc:.4f}")
    print(f"Log Loss:                     {logloss:.4f}")

    print("\nClassification Report:")
    print(
        classification_report(
            y_test,
            y_pred,
            target_names=[
                "Home Win (0)",
                "Draw (1)",
                "Away Win (2)"
            ],
            digits=4,
            zero_division=0
        )
    )

    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))

    print("=" * 70)

    return {
        "Model": model_name,

```

```

        "Accuracy": accuracy,
        "Balanced Accuracy": balanced_acc,
        "Macro Precision": macro_precision,
        "Macro Recall": macro_recall,
        "Macro F1": macro_f1,
        "ROC-AUC Macro": roc_auc,
        "Log Loss": logloss
    }

# =====
# LOGISTIC REGRESSION
# =====

from sklearn.linear_model import LogisticRegression

logistic_model = LogisticRegression(
    max_iter=2000,
    random_state=42
)

logistic_model.fit(X_train_scaled, y_train_final)

lr_results = evaluate_model(
    "Logistic Regression",
    logistic_model,
    X_test_scaled,
    y_test
)

=====
LOGISTIC REGRESSION - TEST SET RESULTS
=====
Accuracy:          0.4470
Balanced Accuracy: 0.4062
Macro Precision:   0.4016
Macro Recall:      0.4062
Macro F1-score:    0.3987
ROC-AUC (Macro):   0.5841
Log Loss:          1.1305

Classification Report:

```

	precision	recall	f1-score	support
Home Win (0)	0.5171	0.6353	0.5701	1072
Draw (1)	0.2614	0.1711	0.2068	637
Away Win (2)	0.4264	0.4122	0.4192	752
accuracy			0.4470	2461
macro avg	0.4016	0.4062	0.3987	2461
weighted avg	0.4232	0.4470	0.4300	2461

Confusion Matrix:

```
[[681 166 225]
 [336 109 192]
 [300 142 310]]
```

```
=====
# =====
# DECISION TREE
# =====
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
print("=" * 70)
print("TRAINING DECISION TREE")
print("=" * 70)
```

```
decision_tree = DecisionTreeClassifier(
    random_state=42
)
```

```
decision_tree.fit(X_train_scaled, y_train_final)
```

```
dt_results = evaluate_model(
    "Decision Tree",
    decision_tree,
    X_test_scaled,
    y_test
)
```

```
=====
TRAINING DECISION TREE
=====
```

```
=====
DECISION TREE - TEST SET RESULTS
=====
```

```
Accuracy:          0.3820
Balanced Accuracy: 0.3659
Macro Precision:   0.3659
Macro Recall:      0.3659
Macro F1-score:    0.3659
ROC-AUC (Macro):   0.5250
Log Loss:          22.2765
```

Classification Report:

	precision	recall	f1-score	support
Home Win (0)	0.4695	0.4599	0.4647	1072
Draw (1)	0.2776	0.2841	0.2808	637
Away Win (2)	0.3505	0.3537	0.3521	752

accuracy			0.3820	2461
macro avg	0.3659	0.3659	0.3659	2461
weighted avg	0.3835	0.3820	0.3827	2461

Confusion Matrix:

```
[[493 273 306]
 [269 181 187]
 [288 198 266]]
```

```
=====
# =====
# RANDOM FOREST
# =====
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
print("=" * 70)
print("TRAINING RANDOM FOREST")
print("=" * 70)
```

```
random_forest = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    n_jobs=-1
)
```

```
random_forest.fit(X_train_scaled, y_train_final)
```

```
rf_results = evaluate_model(
    "Random Forest",
    random_forest,
    X_test_scaled,
    y_test
)
```

```
=====
TRAINING RANDOM FOREST
=====
```

```
=====
RANDOM FOREST - TEST SET RESULTS
=====
```

```
Accuracy:          0.4884
Balanced Accuracy: 0.4181
Macro Precision:   0.4452
Macro Recall:      0.4181
Macro F1-score:    0.3718
ROC-AUC (Macro):   0.6237
Log Loss:          1.0264
```

Classification Report:

	precision	recall	f1-score	support
Home Win (0)	0.4994	0.8209	0.6210	1072
Draw (1)	0.3667	0.0345	0.0631	637
Away Win (2)	0.4695	0.3989	0.4313	752
accuracy			0.4884	2461
macro avg	0.4452	0.4181	0.3718	2461
weighted avg	0.4559	0.4884	0.4187	2461

Confusion Matrix:

```
[[880  22 170]
 [446  22 169]
 [436  16 300]]
```

```
=====
# =====
# XGBOOST / GRADIENT BOOSTING
# =====

from xgboost import XGBClassifier
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    log_loss,
    classification_report,
    confusion_matrix
)

print("=" * 70)
print("TRAINING XGBOOST / GRADIENT BOOSTING")
print("=" * 70)

# -----
# Train XGBoost
# -----

xgb_model = XGBClassifier(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    objective="multi:softprob",
    num_class=3,
    eval_metric="mlogloss",
```

```

        random_state=42,
        n_jobs=-1
    )

xgb_model.fit(
    X_train_scaled,
    y_train_final
)

# -----
# Predictions
# -----

y_pred_xgb = xgb_model.predict(X_test_scaled)
y_prob_xgb = xgb_model.predict_proba(X_test_scaled)

# -----
# Performance metrics
# -----

accuracy = accuracy_score(y_test, y_pred_xgb)

balanced_accuracy = balanced_accuracy_score(
    y_test,
    y_pred_xgb
)

macro_precision = precision_score(
    y_test,
    y_pred_xgb,
    average="macro",
    zero_division=0
)

macro_recall = recall_score(
    y_test,
    y_pred_xgb,
    average="macro",
    zero_division=0
)

macro_f1 = f1_score(
    y_test,
    y_pred_xgb,
    average="macro",
    zero_division=0
)

roc_auc_macro = roc_auc_score(
    y_test,

```

```

        y_prob_xgb,
        multi_class="ovr",
        average="macro"
    )

logloss = log_loss(
    y_test,
    y_prob_xgb
)

# -----
# ONE CLEAN OUTPUT
# -----

print("=" * 70)
print("XGB00ST / GRADIENT BOOSTING - TEST SET RESULTS")
print("=" * 70)

print(f"Accuracy:                {accuracy:.4f}")
print(f"Balanced Accuracy:        {balanced_accuracy:.4f}")
print(f"Macro Precision:           {macro_precision:.4f}")
print(f"Macro Recall:              {macro_recall:.4f}")
print(f"Macro F1-score:            {macro_f1:.4f}")
print(f"ROC-AUC (Macro):           {roc_auc_macro:.4f}")
print(f"Log Loss:                   {logloss:.4f}")

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred_xgb,
        target_names=[
            "Home Win (0)",
            "Draw (1)",
            "Away Win (2)"
        ],
        digits=4,
        zero_division=0
    )
)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_xgb))

print("=" * 70)

```

```

=====
TRAINING XGB00ST / GRADIENT BOOSTING
=====
=====

```

XGBOOST / GRADIENT BOOSTING - TEST SET RESULTS

```
=====
Accuracy:          0.4815
Balanced Accuracy: 0.4231
Macro Precision:   0.4135
Macro Recall:      0.4231
Macro F1-score:    0.3938
ROC-AUC (Macro):   0.6244
Log Loss:          1.0395
```

Classification Report:

	precision	recall	f1-score	support
Home Win (0)	0.5228	0.7481	0.6155	1072
Draw (1)	0.2692	0.0769	0.1197	637
Away Win (2)	0.4483	0.4441	0.4462	752
accuracy			0.4815	2461
macro avg	0.4135	0.4231	0.3938	2461
weighted avg	0.4344	0.4815	0.4354	2461

Confusion Matrix:

```
[[802  65 205]
 [382  49 206]
 [350  68 334]]
```

```
=====
# =====
# NEURAL NETWORK - MULTILAYER PERCEPTRON (MLP)
# =====
```

```
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    log_loss,
    classification_report,
    confusion_matrix
)

print("=" * 70)
print("TRAINING NEURAL NETWORK - MULTILAYER PERCEPTRON")
print("=" * 70)
```

```
# -----
# Train MLP
```

```

# -----
mlp_model = MLPClassifier(
    hidden_layer_sizes=(100, 50),
    activation="relu",
    solver="adam",
    alpha=0.0001,
    learning_rate_init=0.001,
    max_iter=300,
    random_state=42,
    early_stopping=True,
    validation_fraction=0.1,
    n_iter_no_change=20
)

mlp_model.fit(
    X_train_scaled,
    y_train_final
)

# -----
# Predictions
# -----

y_pred_mlp = mlp_model.predict(X_test_scaled)
y_prob_mlp = mlp_model.predict_proba(X_test_scaled)

# -----
# Performance metrics
# -----

accuracy = accuracy_score(
    y_test,
    y_pred_mlp
)

balanced_accuracy = balanced_accuracy_score(
    y_test,
    y_pred_mlp
)

macro_precision = precision_score(
    y_test,
    y_pred_mlp,
    average="macro",
    zero_division=0
)

macro_recall = recall_score(
    y_test,

```

```

        y_pred_mlp,
        average="macro",
        zero_division=0
    )

macro_f1 = f1_score(
    y_test,
    y_pred_mlp,
    average="macro",
    zero_division=0
)

roc_auc_macro = roc_auc_score(
    y_test,
    y_prob_mlp,
    multi_class="ovr",
    average="macro"
)

logloss = log_loss(
    y_test,
    y_prob_mlp
)

# -----
# ONE CLEAN OUTPUT
# -----

print("=" * 70)
print("NEURAL NETWORK (MLP) - TEST SET RESULTS")
print("=" * 70)

print(f"Accuracy:                {accuracy:.4f}")
print(f"Balanced Accuracy:        {balanced_accuracy:.4f}")
print(f"Macro Precision:           {macro_precision:.4f}")
print(f"Macro Recall:              {macro_recall:.4f}")
print(f"Macro F1-score:            {macro_f1:.4f}")
print(f"ROC-AUC (Macro):           {roc_auc_macro:.4f}")
print(f"Log Loss:                   {logloss:.4f}")

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred_mlp,
        target_names=[
            "Home Win (0)",
            "Draw (1)",
            "Away Win (2)"
        ],

```

```

        digits=4,
        zero_division=0
    )
)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_mlp))

print("=" * 70)

```

```

=====
TRAINING NEURAL NETWORK - MULTILAYER PERCEPTRON
=====

```

```

=====
NEURAL NETWORK (MLP) - TEST SET RESULTS
=====

```

```

Accuracy:          0.4575
Balanced Accuracy: 0.3995
Macro Precision:   0.3985
Macro Recall:      0.3995
Macro F1-score:    0.3746
ROC-AUC (Macro):   0.5952
Log Loss:          1.0820

```

```

Classification Report:

```

	precision	recall	f1-score	support
Home Win (0)	0.4975	0.7341	0.5931	1072
Draw (1)	0.2808	0.0895	0.1357	637
Away Win (2)	0.4172	0.3750	0.3950	752
accuracy			0.4575	2461
macro avg	0.3985	0.3995	0.3746	2461
weighted avg	0.4168	0.4575	0.4142	2461

```

Confusion Matrix:

```

```

[[787  75 210]
 [396  57 184]
 [399  71 282]]

```

```

# =====
# KNN IMPUTATION – AUTOMATIC FINAL MODEL COMPARISON
# =====

```

```

import pandas as pd
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    precision_score,

```

```

        recall_score,
        f1_score,
        roc_auc_score,
        log_loss
    )
from IPython.display import display

# -----
# Function to calculate all performance metrics
# -----

def calculate_metrics(model_name, model, X_test, y_test):

    # Generate predictions automatically
    y_pred = model.predict(X_test)

    # Generate probabilities automatically
    y_prob = model.predict_proba(X_test)

    return {
        "Imputation": "KNN",
        "Model": model_name,

        "Accuracy": accuracy_score(
            y_test, y_pred
        ),

        "Balanced Accuracy": balanced_accuracy_score(
            y_test, y_pred
        ),

        "Macro Precision": precision_score(
            y_test,
            y_pred,
            average="macro",
            zero_division=0
        ),

        "Macro Recall": recall_score(
            y_test,
            y_pred,
            average="macro",
            zero_division=0
        ),

        "Macro F1": f1_score(
            y_test,
            y_pred,
            average="macro",

```

```

        zero_division=0
    ),

    "ROC-AUC Macro": roc_auc_score(
        y_test,
        y_prob,
        multi_class="ovr",
        average="macro"
    ),

    "Log Loss": log_loss(
        y_test,
        y_prob
    )
}

# =====
# CALCULATE RESULTS
# =====

knn_results = []

# -----
# Logistic Regression
# -----

knn_results.append(
    calculate_metrics(
        "Logistic Regression",
        logistic_model,
        X_test_scaled,
        y_test
    )
)

# -----
# Decision Tree
# -----

knn_results.append(
    calculate_metrics(
        "Decision Tree",
        decision_tree,
        X_test_scaled,
        y_test
    )
)

```

```

# -----
# Random Forest
# -----

knn_results.append(
    calculate_metrics(
        "Random Forest",
        random_forest,
        X_test_scaled,
        y_test
    )
)

# -----
# XGBoost
# -----

knn_results.append(
    calculate_metrics(
        "XGBoost",
        xgb_model,
        X_test_scaled,
        y_test
    )
)

# -----
# Neural Network / MLP
# -----

knn_results.append(
    calculate_metrics(
        "MLP",
        mlp_model,
        X_test_scaled,
        y_test
    )
)

# =====
# CREATE FINAL COMPARISON TABLE
# =====

knn_results_df = pd.DataFrame(knn_results)

```

```
print("=" * 120)
print("KNN IMPUTATION – FINAL MODEL PERFORMANCE COMPARISON")
print("=" * 120)
```

```
display(knn_results_df.round(4))
```

```
=====
=====
KNN IMPUTATION – FINAL MODEL PERFORMANCE COMPARISON
=====
=====
```

	Imputation	Model	Accuracy	Balanced Accuracy	\
0	KNN	Logistic Regression	0.4470	0.4062	
1	KNN	Decision Tree	0.3820	0.3659	
2	KNN	Random Forest	0.4884	0.4181	
3	KNN	XGBoost	0.4815	0.4231	
4	KNN	MLP	0.4575	0.3995	

	Macro Precision	Macro Recall	Macro F1	ROC-AUC	Macro	Log Loss
0	0.4016	0.4062	0.3987		0.5841	1.1305
1	0.3659	0.3659	0.3659		0.5250	22.2765
2	0.4452	0.4181	0.3718		0.6237	1.0264
3	0.4135	0.4231	0.3938		0.6244	1.0395
4	0.3985	0.3995	0.3746		0.5952	1.0820